Introduction: Lexicon as Data and Manuscript as Build Artifact

Introduction: Lexicon as Data and Manuscript as Build Artifact I

Mad Lib style generation is usually treated as a toy because it foregrounds the visible blank rather than the source of the replacement. In a research pipeline, the blank is not the hard part. The hard part is making every replacement reviewable, deterministic, and honest about what it can support. `template_madlib` turns that constraint into the subject of the exemplar.

The project treats nouns such as pipeline, protocol, section, lexicon and verbs such as hydrate, condition, bind, bind as versioned data. Changing a lexicon entry is therefore closer to changing an input table than editing prose in place. That distinction matters because the generated manuscript can be rerendered, diffed, and validated without asking the reader to trust an invisible drafting session.

Introduction: Lexicon as Data and Manuscript as Build Artifact II

The introduction is configured to separate playful Mad Lib syntax from research claims, identify drift between prose and source data, frame configuration as an inspectable dataset, and position conditional prose as a reproducibility problem. Those moves keep the manuscript from pretending to be an open-ended language model. It is a bounded template: authors declare categories, slots, section switches, method steps, and claim boundaries; source code transforms those declarations into manuscript bodies and evidence tables.

This exemplar is useful for protocols, educational scaffolds, review forms, templated reports, and other documents where conditional text is unavoidable but should never become untraceable. The same pattern can be extended with domain-specific validators while leaving the shared rendering infrastructure untouched.

Contribution Ledger I

Claim	Boundary
A Mad Lib manuscript can remain reproducible when the lexicon is treated as data.	Local exemplar claim; no live DOI or standalone release implied.
Conditional IMRAD section bodies can be rendered without shared renderer changes.	Local exemplar claim; no live DOI or standalone release implied.
Large-grain manuscript variables can preserve author-readable source files while still producing a complete manuscript.	Local exemplar claim; no live DOI or standalone release implied.
Token provenance can connect playful lexical substitution to serious publication hygiene.	Local exemplar claim; no live DOI or standalone release implied.
A generated Methods section can be methodologically useful when protocol rows, phases, figures, and validation gates share one config-owned source.	Local exemplar claim; no live DOI or standalone release implied.
Configured-field origin tracking can make loader defaults visible enough for reviewers and downstream forks.	Local exemplar claim; no live DOI or standalone release implied.
Pipeline-owned output regeneration can keep PDF, HTML, slides, data, reports, and copied deliverables aligned without hand-editing generated artifacts.	Local exemplar claim; no live DOI or standalone release implied.
A generated-method exemplar can make review handoff auditable when the review packet includes source config, data artifacts, figures, validation results, and copy statistics.	Local exemplar claim; no live DOI or standalone release implied.
Fork migration guidance can reduce overclaiming when it names the source, test, validator, and evidence surfaces that must change before domain use.	Local exemplar claim; no live DOI or standalone release implied.